Trojan Market - Detailed Design

Database Schema

The database will have five main tables: User, Posting, Chat Sessions, Messages, and Transactions. All primary keys will automatically increment to ensure unique records.

User Table:
This will store account credentials, verification status, and cumulative review scores.

Field	Data Type	Constraints	Description
userID	INT	PRIMARY KEY	Unique identifier for each user.
username	VARCHAR(25)	NOT NULL, UNIQUE	User's display and login name.
password	VARCHAR(25)	NOT NULL	Account password.
email	VARCHAR(50)	NOT NULL, UNIQUE	Primary contact and verification email.
isVerified	BOOLEAN	DEFAULT FALSE	Indicates if the user has a verified account.

review	INT	DEFAULT 0	Cumulative rating score from other users.

Posting Table:
This will contain the data for individual marketplace listings.

Field	Data Type	Constraints	Description
postID	INT	PRIMARY KEY	Unique identifier for the listing.
sellerID	INT	FOREIGN KEY	References User.userID.
title	VARCHAR(50)	NOT NULL	Brief headline of the item.
description	VARCHAR(250)		Detailed description of the item.
category	ENUM(...)	NOT NULL	Item category (e.g., 'Books', 'Tech').

status	ENUM(...)	NOT NULL	'AVAILABLE', 'PENDING', or 'SOLD'.
price	DECIMAL(10,2)	NOT NULL	Asking price for the item.
postTime	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Date and time the post was created.

Chat Sessions Table:
This will be a high-level container that links a buyer and seller to a specific listing.

Field	Data Type	Constraints	Description
sessionID	INT	PRIMARY KEY	Unique identifier for the chat.
postID	INT	FOREIGN KEY	References Posting.postID.
buyerID	INT	FOREIGN KEY	References User.userID.
sellerID	INT	FOREIGN KEY	References User.userID.

Messages Table:
This will store the communications within established chat sessions.

Field	Data Type	Constraints	Description
messageID	INT	PRIMARY KEY	Unique identifier for the message.
sessionID	INT	FOREIGN KEY	References ChatSessions.sessionID.
senderID	INT	FOREIGN KEY	References User.userID.
messageText	TEXT	NOT NULL	The content of the message.
messageTime	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Time the message was sent.

Transactions Table:
This will act as a ledger of completed sales and contain user purchase history.

Field	Data Type	Constraints	Description

transactionID	INT	PRIMARY KEY	Unique identifier for the record.
postID	INT	FOREIGN KEY	References Posting.postID.
buyerID	INT	FOREIGN KEY	References User.userID.
sellerID	INT	FOREIGN KEY	References User.userID.
price	DECIMAL(10,2)	NOT NULL	The final price at the time of sale.
transactionTime	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Date and time of completion.

Authentication System

The authentication system manages user identity, account security, and access control for Trojan Market. It will use the USC Single Sign-on system along with Duo Multi-Factor Authentication to ensure that only verified USC students can access the platform.

Class Design

AuthController.java: handles incoming authentication requests and delegates logic to the AuthService

- handleSSOCallback(token)
    - Receives the authenticated SSO token, extracts user attributes, and delegates to the authentication service
- getCurrentUser(authentication)
    - Returns the currently authenticated user's information.
- logout(response)
    - Clears tokens from client

AuthService.java:

- handleSSOLogin(User)
    - Main entry point for authentication logic. Validates USC identity, creates or retrieves user, and generates tokens
- createUserIfNotExists(User)
    - Checks if a user exists by email. If not, create a new user record with verified status
- validateUSCEmail(email)
    - Ensures the email belongs to the USC domain (@usc.edu).
- isBannedUser(userID)
    - Checks if a user has 10 or more reports and flags them as restricted.

Authentication Flow

1. User clicks "Sign In with USC" which redirects them to USC SSO login
2. User enters credentials and completes DUO authentication
3. USC redirects back to the backend with an authentication token
4. AuthController.handleSSOCallback() receives the token and extracts user information
5. AuthService.handleSSOLogin() validates the user, creates an account if needed, and generates a JWT
6. JWT is returned to the client and stored

7. The following requests include this JWT which will validate the user and allow access

Messaging System

Overview

The messaging system enables registered USC users to communicate directly with sellers about specific listings. Each conversation is tied to a single posting so both parties always have context on what is being discussed. Messaging is implemented using a combination of REST API for session management and WebSocket connections for real-time message delivery.

Access Rules

Guest users and high-risk users (10+ reports) cannot start a chat, send messages, or view history. Only registered users have the ability to start a chat and send messages. They can view the history of only their own chats.

The access is enforced on the backend. The frontend disables buttons as a convenience, but the Spring Security layer returns HTTP 403 regardless if a blocked user calls the endpoint directly.

Class Design

ChatController.java: exposes the REST endpoints for session management and message history. Also serves as the WebSocket message handler via @MessageMapping.

- POST /api/v1/chat/sessions
    - Gets or creates a chat session for a given postID and the authenticated buyer
- GET /api/v1/chat/sessions
    - returns all sessions the authenticated user is part of
- GET /api/v1/chat/sessions/{sessionID}/messages
    - returns the full message history for a session
- @MessageMapping("/chat.send") // handleMessage(ChatMessageDTO msg)
    - receives an inbound WebSocket STOMP frame and delegates to ChatService.sendMessage()

ChatService.java: contains all business logic for the messaging system

- getOrCreateSession(int postID, int buyerID) : ChatSession DTO
    - Checks if a session already exists for this postID + buyerID pair. If not, it creates one. Throws ForbiddenException if the caller is a high-risk user or guest.
- sendMessage(int sessionID, int senderID, String text) : Message DTO

        - Persists the message to the Messages table, broadcasts it to the WebSocket topic for the session, and triggers a notification to the recipient.
- getHistory(int sessionID, int callerID) : List<MessageDTO>
    - Returns all messages for the session in chronological order. Throws ForbiddenException if the caller is not a participant in the session.
- validateParticipant(int sessionID, int userID) : void
    - Helper method. Throws ForbiddenException if userID is neither the buyerID nor the sellerID.

ChatMessageDTO.java: data transfer object for inbound WebSocket messages
- int sessionID: which session the message belongs to
- int senderID: populated server-side from JWT, not trusted from client
- String messageText: the message body

WebSocket Architecture

Real-time delivery is handled using Spring WebSocket with the STOMP sub-protocol over SockJS. The Spring message broker runs on a dedicated thread pool separate from the HTTP request threads, meaning message delivery is non-blocking and concurrent across multiple active sessions.

Channels
- /app/chat.send
    - Client → Server
    - Client sends a new message to a session
- /topic/chat/{sessionID}
    - Server → Clients
        - Server broadcasts the message to both participants
- /user/{userID}/queue/notifications
    - Server → Client
    - Delivers a new-message notification to the recipient
- /user/{userID}/queue/unread
    - Server → Client
    - Updates the unread message badge count in the navigation bar

When the page loads, the frontend opens a single persistent WebSocket connection. When a user opens a chat thread, the client subscribes to */topic/chat/{sessionID}*. Incoming messages on that topic are appended to the thread in real time without any polling.

Message Delivery Sequence

1. Buyer clicks “Message Seller” on a listing page → frontend calls *POST /api/v1/chat/sessions* with the *postID*.
2. ChatService.getOrCreateSession() returns an existing session or creates a new one. Guest and high-risk users are rejected here with HTTP 403.
3. Frontend subscribes to the WebSocket topic */topic/chat/{sessionID}*.
4. User types a message and presses “Send”→ frontend sends a STOMP frame to */app/chat.send* with *{sessionID, messageText}*.
5. ChatController.handleMessage() receives the frame. The *senderID* is extracted from the authenticated JWT. It is not trusted from the client payload.
6. ChatService.sendMessage() persists the *Message* row to the database.
7. The message is broadcasted to */topic/chat/{sessionID}*. Both the sender and recipient receive it instantly via their open subscriptions.
8. NotificationService.push() sends a *NEW_MESSAGE* notification to the recipient’s private queue.
9. The frontend’s message list updates in real time on receipt of the WebSocket event.

Posting System

The posting system allows registered USC users to list items for sale, transitioning them through a structured lifecycle from "Available" to "Sold". It serves as the data foundation for the search and messaging systems, ensuring that every interaction is tied to a specific listing.

createPosting()

This function handles the logic for a user submitting a new listing to the marketplace.

- Parameters:
  - sellerID (INT): Extracted from the authenticated session/JWT to link the post to the correct user.
  - title (VARCHAR): The headline for the item.
  - description (VARCHAR): Detailed information about the item.
  - category (ENUM): Must match one of the predefined categories (e.g., 'Books', 'Tech').
  - price (DECIMAL): The initial asking price.
  - condition (ENUM): The physical state of the item (e.g., 'New', 'Good').
  - photos (LIST): A collection of image files or URLs (minimum one required).
- Functionality:
  - Validation: Verifies that the user is registered and not a "high-risk" user (10+ reports), as guests cannot interact with listings.

    - Persistence: Inserts a new record into the Posting table with a status of AVAILABLE.
    - Defaulting: Automatically sets the postID via auto-increment and assigns the current server time to postTime.
    - Notification: Triggers the NotificationSystem to alert users who may have favorited similar items or categories.

editPosting()

This function allows a seller to update the details or status of an existing listing they own.

- Parameters:
    - postID (INT): The unique identifier of the listing to be modified.
    - callerID (INT): The ID of the user attempting the edit (used for authorization).
    - updatedFields (MAP): A collection of key-value pairs for the fields being changed (e.g., price, description, status).
- Functionality:
    - Authorization Check: Uses a helper method similar to validateParticipant() to ensure the callerID matches the sellerID associated with that postID.
    - Status Update: Primarily used to transition an item's status between AVAILABLE, PENDING, and SOLD.
    - Data Integrity: Updates the Posting table while ensuring the price remains a valid decimal and the category stays within the allowed ENUM values.
    - Synchronization: If the status is changed to SOLD, the system may trigger the creation of a record in the Transactions table to act as a ledger for purchase history.

Search System

Overview

The search manager class provides all search and filtering functionality for Trojan Market. It accesses Posting and User tables in the MySQL database TrojanMarket using the JDBC MySQL driver. The class returns only listings with a status of AVAILABLE. Guests and registered users can use the search system.

searchPostings()

This function can take the following parameters:

Keywords: any string can be entered. It parses all text from listings with a priority of relevance to

return.
Category: matched string to item categories (ex: “Books”, “Furniture”, etc.)
minPrice/maxPrice: price bounds
sortBy: price, relevance, new listings

Returns a List where each object has PostingID, title, price, photo, sellerID, sellerRating, and status (always AVAILABLE).

getCategories()

Returns a list of category values pulled from the Posting table. Frontend has a dropdown filter for categories. Categories are: Books, Clothing & Accessories, Electronics, Furniture, School Supplies, Tickets, and Other.

getPostingDetail()

This gives all info on a posting: postID, title, description, category, price, condition, postTime, status, Photo, seller’s username, seller’s rating

Search Algorithm

When the user searches, the input is taken as a string and then cleaned. It is trimmed, lowercased, and stripped of special characters. Then, by whitespace, it is split into an array of individual strings which are taken as keywords. Then, there are 2 phases

Phase 1 - Database Retrieval

One SQL query pulls all status == AVAILABLE postings where the keyword appears at least once in the title or description

Phase 2 - Relevance Scoring

After the set of results is pulled, a score is calculated for each posting as following:

+3 - keyword has a whole word match in the title
+2 - keyword has a substring match in the title
+1 - keyword has a substring match in the description

Total score for each posting is the sum of points across all search keywords. The results are then displayed in order of score highest to lowest.

Scoring

Results are sorted with the sortBy parameter by the following:

relevant - descending by score as shown above
price_asc - ascending by price
price_dec - descending by price
Newest - descending by postTime

Error Handling

Empty results: If no listings match the query and filters, a message like “No listings found” will be displayed.
Invalid price range: If minprice exceeds max price, the user will be prompted to change those values

Save/Bookmark System

The StatsManager class will access the Transactions, Messages, Postings and User tables in the MySQL database TrojanMarket using the JDBC MySQL driver.

The getPastPurchases() method will take the userID as a parameter and will retrieve all transactions where the user is the buyer. This includes a list of purchased items including the title, price, seller, and chat history associated with the item.

The getSoldItems() method will take the userID as a parameter and will retrieve all transactions where the user has sold an item. This includes a list of purchased items including the title, price, seller, and chat history associated with the item.

The getSavedPostings() method will take the userID as a parameter and will retrieve all postings that the user liked/bookmarked. This includes the title, price, and seller of the item.Additionally this method will only be available to registered users.

The getChat() method will take the postID and userID as parameters and will return the message history associated with the postID.

The removeSavedPosting() method will take the userID and postID as parameters and will remove the selected posting from the user’s saved list.

Notifications System

The notifications system alerts registered users of marketplace activity relevant to their accounts. The NotificationSystem class will use the Notifications table and User table.

getNotifications() will take the parameter: userID and will return all notifications for that user, with the most recent being at the top of the messages. The notification type will list related, favorited items, and messages between the buyer and seller.

getUnreadCount() will take the parameter: userID and will return the count of unread notifications for the number next to the bell icon.

createNotification() will take a receiverID, type, relatedPostID, and relatedSessionID as parameters. This method will insert a new notification into the database. It will check the user's notification preferences before inserting.

markRead() will take the notificationID and will update the isRead field to true.

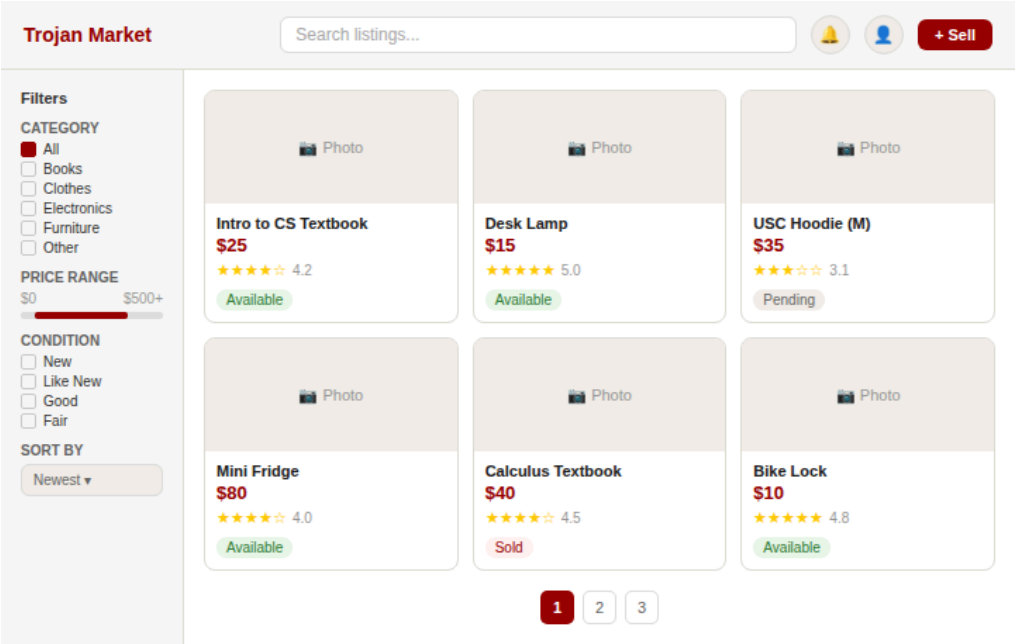
clearAll() will take a userID and delete all notifications.

updatePreferences() will take a userID and a preferences map as parameters and will update the user's notification preferences to show which notification settings they want.

A user must be registered to receive notifications, since guests cannot interact with listings or other users.

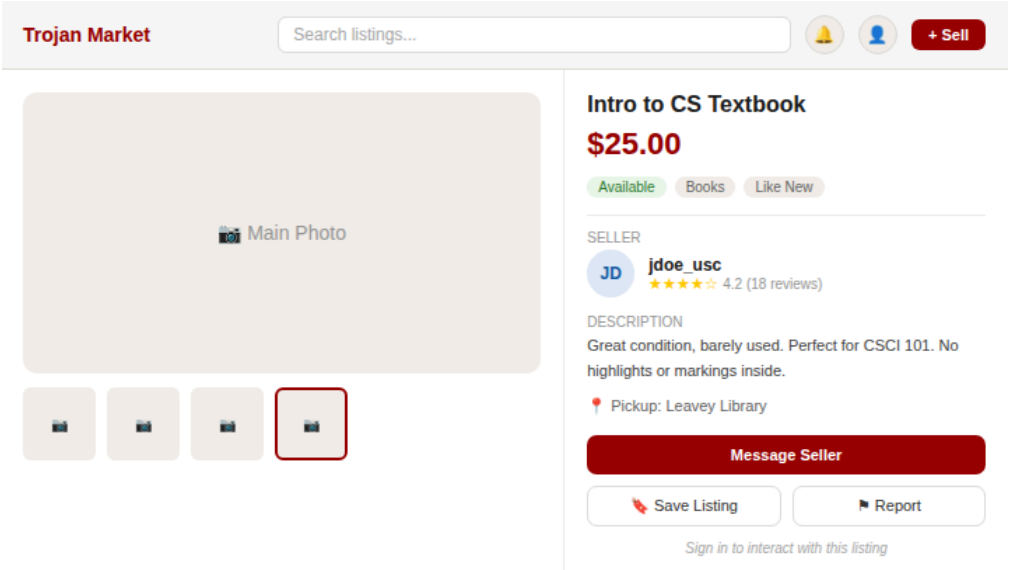
GUI Design

Home/Browse Page



Navigation Bar includes the Trojan Market logo, a persistent search bar, notification bell, profile avatar, and a “Sell” button. The lift sidebar contains category checkboxes, a price range slider, condition filter, and sort dropdown. The Main area displays a responsive grid of PostingCards showing photo, title, price, seller rating, and status badge. Page select appears at the bottom.

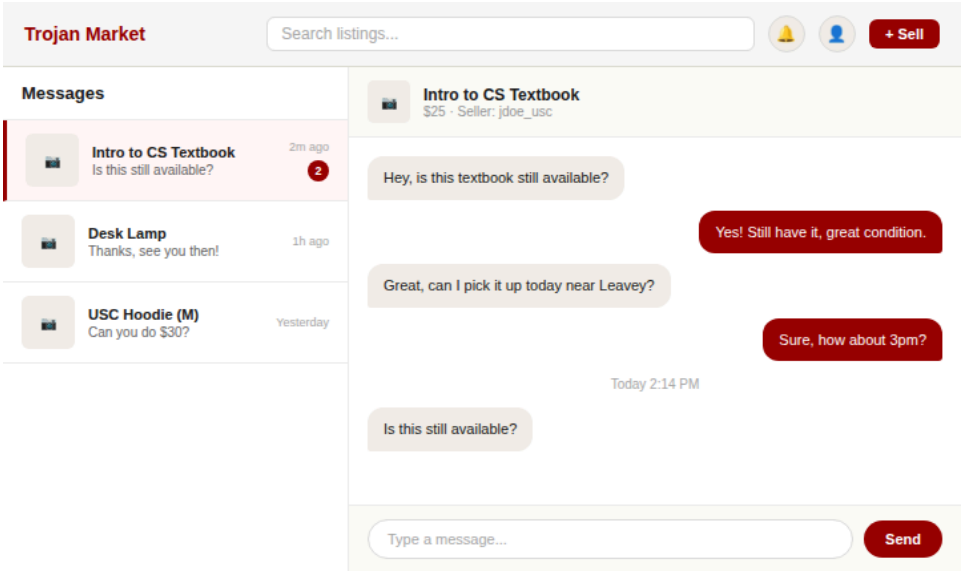
Listing Detail Page



Left column shows an image carousel (main photo + thumbnail strip, up to 5 images). Right column shows title, price, status/category/condition badges, seller chip (avatar + username + star rating), description, pickup location, and action buttons. “Message Seller” is disabled for guests

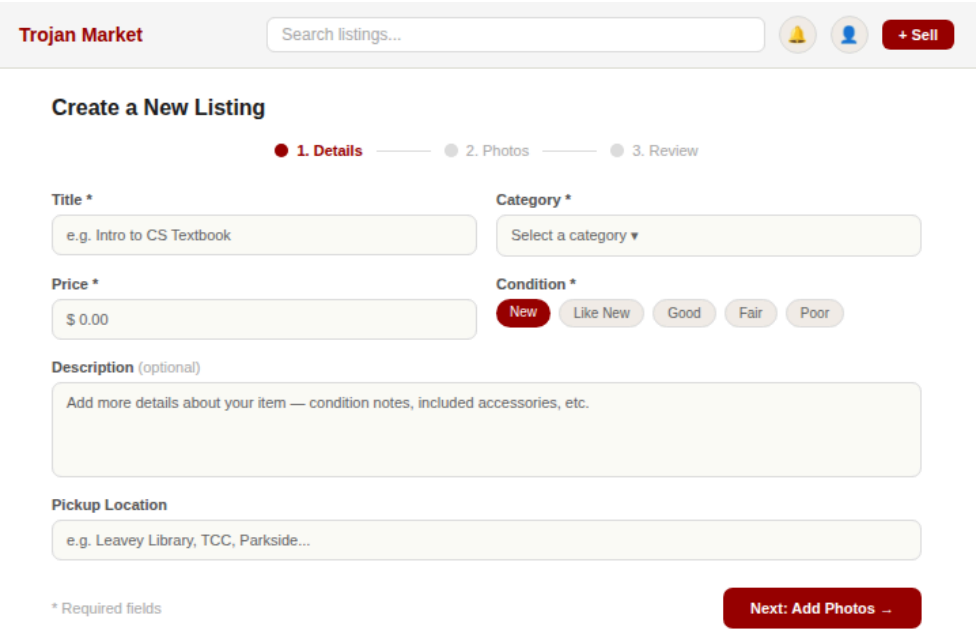
and high-risk users. Instead, they see a “Sign In to Interact” prompt instead. Save and Report are also actions below.

Messages Page



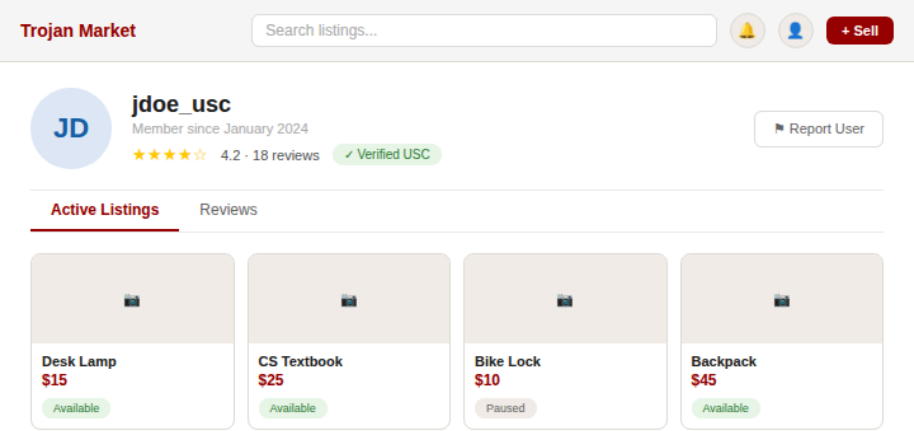
Two-panel layout. Left panel: session list sorted by most recent, showing listing thumbnail, other party’s username, last message preview, timestamp, and unread count badge. Active session is highlighted. Right panel: listing info banner at the top (thumbnail + title + seller), followed by the chronological chat thread with sent messages on the right in red and the received on the left in gray. Text input and Send button are pinned to the bottom.

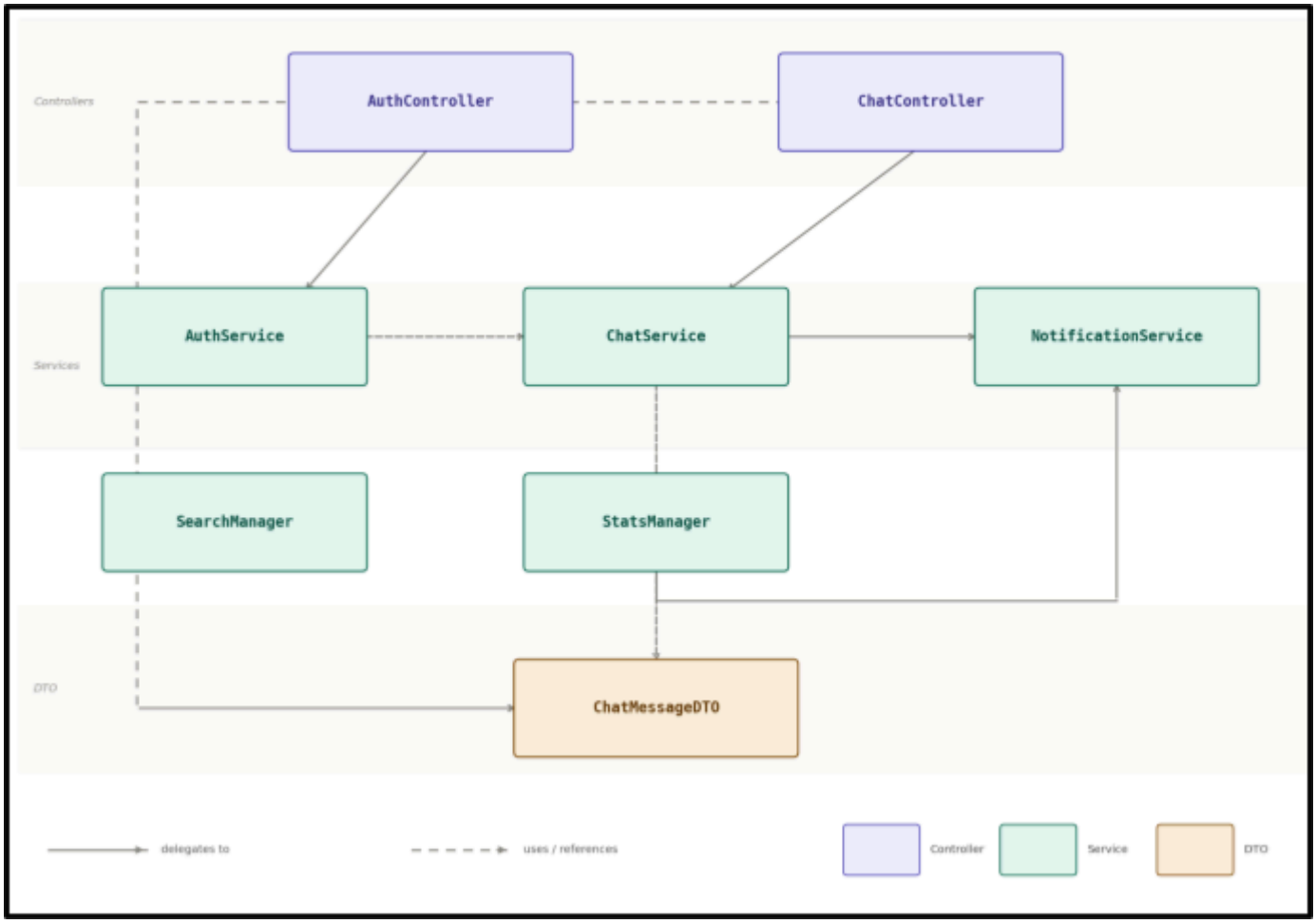
Create Posting Page



This form has three steps. Step 1 (details): title, category dropdown, price, condition cells, optional description, pickup location. Step 2 (photos): drag-and-drop image uploader with the preview thumbnails, minimum one photo required. Step 3 (review): a read-only PostingCard preview of the listing with “Back” and “Publish” buttons. The step progress indicator at the top shows which step is active.

User Profile Page



Header section: avatar (initials if none uploaded), username, member-since date, star rating with review count, and “Verified USC” badge. A “Report” button sits at the top-right. Two tabs below the divider: “Active Listings” (PostingCard for this seller’s available items) and “Reviews” (list of ReviewCards with reviewer name, star rating, comment, and date). High-risk users display a warning badge in the header instead of the verified badge.

Class Diagram